Gateway HTTP 게이트웨이

개요

IPFS Gateway는 HTTP 프로토콜을 통해 IPFS 콘텐츠에 접근할 수 있게 해주는 브리지 역할을 합니다. 웹 브라우저나 일반적인 HTTP 클라이언트가 IPFS 네트워크의 데이터를 읽을 수 있도록 HTTP API를 제공합니다.

Gateway는 다양한 형태로 구현될 수 있으며, 신뢰할 수 있는(Trustless) 게이트웨이부터 완전한 기능을 제공하는 게이트웨이까지 다양한 설정이 가능합니다.

HTTP 게이트웨이 개념과 종류

1. Path Gateway (경로 게이트웨이)

가장 기본적인 형태로, URL 경로를 통해 IPFS 콘텐츠에 접근합니다.

```
https://gateway.example.com/ipfs/QmHash...

https://gateway.example.com/ipns/example.com
```

2. Subdomain Gateway (서브도메인 게이트웨이)

각 CID나 IPNS 이름을 서브도메인으로 사용하여 Origin 격리를 제공합니다.

```
https://QmHash....ipfs.gateway.example.com/

https://example-com.ipns.gateway.example.com/
```

3. DNSLink Gateway (DNS링크 게이트웨이)

DNS TXT 레코드를 통해 도메인을 IPFS 콘텐츠에 연결합니다.

```
https://example.com → /ipfs/QmHash... (DNS TXT 레코드 참조)
```

4. Trustless Gateway (신뢰할 수 없는 게이트웨이)

암호학적으로 검증 가능한 응답만 제공하여 게이트웨이를 신뢰할 필요가 없습니다.

기본 게이트웨이 구현 방법

간단한 게이트웨이 서버

```go
package main


import (
"context"
"log"
"net/http"
"github.com/ipfs/boxo/blockservice"
"github.com/ipfs/boxo/blockstore"
"github.com/ipfs/boxo/exchange/offline"
"github.com/ipfs/boxo/gateway"
"github.com/ipfs/go-datastore"
dsync  "github.com/ipfs/go-datastore/sync"
```

```
)

func main() {
ctx := context.Background()
// 블록스토어 설정
ds := dsync.MutexWrap(datastore.NewMapDatastore())
bs := blockstore.NewBlockstore(ds)
// 블록 서비스 생성 (오프라인 모드)
blockService := blockservice.New(bs, offline.Exchange(bs))
// Gateway 백엔드 생성
backend, err := gateway.NewBlocksBackend(blockService)
if err != nil {
log.Fatal(err)
}
// Gateway 설정
config := gateway.Config{
DeserializedResponses: true, // 역직렬화된 응답 허용
NoDNSLink: false, // DNSLink 지원
}
// Gateway 핸들러 생성
gwHandler := gateway.NewHandler(config, backend)
// HTTP 서버 설정
mux := http.NewServeMux()
```

```
mux.Handle("/ipfs/", gwHandler)

mux.Handle("/ipns/", gwHandler)

log.Println("Gateway 서버 시작: http://localhost:8080")

log.Fatal(http.ListenAndServe(":8080", mux))

}
```

CAR 파일 기반 게이트웨이

```
package main


import (

"context"

"io"

"log"

"net/http"

"os"

"github.com/ipfs/boxo/blockservice"

"github.com/ipfs/boxo/exchange/offline"

"github.com/ipfs/boxo/gateway"

"github.com/ipfs/go-cid"

carblockstore  "github.com/ipld/go-car/v2/blockstore"

)
```

```go
func  createCARGateway(carFilePath  string) (http.Handler, []cid.Cid, error) {
// CAR 파일 열기
r, err := os.Open(carFilePath)
if  err != nil {
return  nil, nil, err
}
// CAR 블록스토어 생성
bs, err := carblockstore.NewReadOnly(r, nil)
if  err != nil {
r.Close()
return  nil, nil, err
}
// 루트 CID들 가져오기
roots, err := bs.Roots()
if  err != nil {
return  nil, nil, err
}
// 블록 서비스 생성
blockService := blockservice.New(bs, offline.Exchange(bs))
// Gateway 백엔드 생성
backend, err := gateway.NewBlocksBackend(blockService)
if  err != nil {
return  nil, nil, err
```

```
}
// Gateway 설정
config := gateway.Config{
DeserializedResponses: true,
NoDNSLink: false,
}
// Gateway 핸들러 생성
gwHandler := gateway.NewHandler(config, backend)
// HTTP 멀티플렉서 설정
mux := http.NewServeMux()
mux.Handle("/ipfs/", gwHandler)
mux.Handle("/ipns/", gwHandler)
return mux, roots, nil
}


func main() {
handler, roots, err := createCARGateway("example.car")
if err != nil {
log.Fatal(err)
}
log.Println("CAR 기반 Gateway 시작: http://localhost:8080")
for _, root := range roots {
log.Printf("루트 CID: http://localhost:8080/ipfs/%s", root.String())
```

```
}
log.Fatal(http.ListenAndServe(":8080", handler))
}
```

프록시 게이트웨이 구현

HTTP 프록시 게이트웨이

```
package main

import (
"context"
"fmt"
"io"
"log"
"net/http"
"net/url"
"strings"
"time"
"github.com/ipfs/boxo/gateway"
"github.com/ipfs/boxo/path"
)
```

```go
// ProxyBackend는 다른 IPFS 게이트웨이로 요청을 프록시하는 백엔드입니다
type ProxyBackend struct {
gatewayURL  string
client *http.Client
}

func  NewProxyBackend(gatewayURL  string) *ProxyBackend {
return &ProxyBackend{
gatewayURL: strings.TrimSuffix(gatewayURL, "/"),
client: &http.Client{
Timeout: 30 * time.Second,
},
}
}

func (p *ProxyBackend) Get(ctx context.Context, immutablePath path.ImmutablePath
// 프록시 URL 구성
proxyURL := p.gatewayURL + immutablePath.String()
// HTTP 요청 생성
req, err := http.NewRequestWithContext(ctx, "GET", proxyURL, nil)
if  err != nil {
return gateway.ContentPathMetadata{}, nil, err
}
```

```
// Range 헤더 설정 (필요한 경우)
if len(ranges) > 0 {
r := ranges[0]
var rangeHeader string
if r.To != nil {
rangeHeader = fmt.Sprintf("bytes=%d-%d", r.From, *r.To)
} else {
rangeHeader = fmt.Sprintf("bytes=%d-", r.From)
}
req.Header.Set("Range", rangeHeader)
}
// 요청 실행
resp, err := p.client.Do(req)
if err != nil {
return gateway.ContentPathMetadata{}, nil, err
}
if resp.StatusCode != http.StatusOK && resp.StatusCode != http.StatusPartialCon
resp.Body.Close()
return gateway.ContentPathMetadata{}, nil, fmt.Errorf("프록시 요청 실패: %d", resp
}
// 메타데이터 구성
metadata := gateway.ContentPathMetadata{
ContentType: resp.Header.Get("Content-Type"),
```

```
}

// 응답 생성

response := gateway.NewGetResponseFromReader(resp.Body, resp.ContentLength)

return  metadata, response, nil

}


func (p *ProxyBackend) GetAll(ctx context.Context, immutablePath path.ImmutableP

// GetAll 구현 (필요에 따라)

return gateway.ContentPathMetadata{}, nil, fmt.Errorf("GetAll not implemented in

}


func (p *ProxyBackend) GetBlock(ctx context.Context, immutablePath path.Immutabl

// GetBlock 구현 (필요에 따라)

return gateway.ContentPathMetadata{}, nil, fmt.Errorf("GetBlock not implemented

}


func (p *ProxyBackend) Head(ctx context.Context, immutablePath path.ImmutablePat

// HEAD 요청 프록시

proxyURL := p.gatewayURL + immutablePath.String()

req, err := http.NewRequestWithContext(ctx, "HEAD", proxyURL, nil)

if  err != nil {

return gateway.ContentPathMetadata{}, nil, err

}
```

```
resp, err := p.client.Do(req)
if  err != nil {
return gateway.ContentPathMetadata{}, nil, err
}
defer  resp.Body.Close()
if  resp.StatusCode != http.StatusOK {
return gateway.ContentPathMetadata{}, nil, fmt.Errorf("프록시 HEAD 요청 실패: %d",
}
metadata := gateway.ContentPathMetadata{
ContentType: resp.Header.Get("Content-Type"),
}
headResp := gateway.NewHeadResponseForFile(nil, resp.ContentLength)
return  metadata, headResp, nil
}


// 기타 필수 메서드들 구현...
func (p *ProxyBackend) ResolvePath(ctx context.Context, immutablePath path.Immut
return gateway.ContentPathMetadata{}, nil
}


func (p *ProxyBackend) GetCAR(ctx context.Context, immutablePath path.ImmutableP
return gateway.ContentPathMetadata{}, nil, fmt.Errorf("GetCAR not implemented in
}
```

```go
func (p *ProxyBackend) IsCached(ctx context.Context, p path.Path) bool {
return false
}

func (p *ProxyBackend) GetIPNSRecord(ctx context.Context, c cid.Cid) ([]byte, er
return  nil, fmt.Errorf("GetIPNSRecord not implemented in proxy backend")
}

func (p *ProxyBackend) ResolveMutable(ctx context.Context, p path.Path) (path.Im
return  nil, 0, time.Time{}, fmt.Errorf("ResolveMutable not implemented in proxy
}

func (p *ProxyBackend) GetDNSLinkRecord(ctx context.Context, hostname  string) (
return  nil, fmt.Errorf("GetDNSLinkRecord not implemented in proxy backend")
}

func  main() {
// 업스트림 게이트웨이 설정
backend := NewProxyBackend("https://ipfs.io")
// Gateway 설정
config := gateway.Config{
DeserializedResponses: true,
```

```
NoDNSLink: false,
}
// Gateway 핸들러 생성
gwHandler := gateway.NewHandler(config, backend)
// HTTP 서버 설정
mux := http.NewServeMux()
mux.Handle("/ipfs/", gwHandler)
mux.Handle("/ipns/", gwHandler)
log.Println("프록시 Gateway 시작: http://localhost:8080")
log.Println("업스트림: https://ipfs.io")
log.Fatal(http.ListenAndServe(":8080", mux))
}
```

설정 옵션과 보안 고려사항

고급 Gateway 설정

```
func  createAdvancedGateway(backend gateway.IPFSBackend) http.Handler {
config := gateway.Config{
// 역직렬화된 응답 허용 (보안상 주의 필요)
DeserializedResponses: true,
// DNSLink 비활성화 (보안 강화)
NoDNSLink: false,
```

```
// HTML 오류 페이지 비활성화
DisableHTMLErrors: false,
// 검색 타임아웃 설정
RetrievalTimeout: 30 * time.Second,
// 최대 동시 요청 수 제한
MaxConcurrentRequests: 1000,
// 공개 게이트웨이 설정
PublicGateways: map[string]*gateway.PublicGateway{
"localhost": {
Paths: []string{"/ipfs", "/ipns"},
UseSubdomains: true,
NoDNSLink: false,
InlineDNSLink: true,
DeserializedResponses: true,
},
"example.com": {
Paths: []string{"/ipfs"},
UseSubdomains: false,
NoDNSLink: true, // 보안상 DNSLink 비활성화
DeserializedResponses: false, // Trustless 모드
},
},
// 사용자 정의 메뉴 항목
```

```
Menu: []assets.MenuItem{
{
URL: "https://docs.ipfs.tech",
Title: "IPFS 문서",
},
{
URL: "https://github.com/ipfs/boxo",
Title: "Boxo 라이브러리",
},
},
}
// Gateway 핸들러 생성
gwHandler := gateway.NewHandler(config, backend)
// HTTP 멀티플렉서 설정
mux := http.NewServeMux()
mux.Handle("/ipfs/", gwHandler)
mux.Handle("/ipns/", gwHandler)
// 메트릭 엔드포인트 추가
mux.Handle("/debug/metrics/prometheus", promhttp.Handler())
// 호스트명 핸들러로 래핑 (서브도메인 지원)
handler := gateway.NewHostnameHandler(config, backend, mux)
// CORS 헤더 추가
handler = gateway.NewHeaders(map[string][]string{
```

```
"Access-Control-Allow-Origin": {"*"},
"Access-Control-Allow-Methods": {"GET, HEAD, OPTIONS"},
"Access-Control-Allow-Headers": {"Content-Type, Range"},
}).ApplyCors().Wrap(handler)
return  handler
}
```

보안 강화 설정

```
func  createSecureGateway(backend gateway.IPFSBackend) http.Handler {
config := gateway.Config{
// Trustless 모드 - 암호학적으로 검증 가능한 응답만 허용
DeserializedResponses: false,
// DNSLink 완전 비활성화
NoDNSLink: true,
// 빠른 타임아웃으로 DoS 공격 방지
RetrievalTimeout: 10 * time.Second,
// 동시 요청 수 제한으로 리소스 보호
MaxConcurrentRequests: 100,
// HTML 오류 페이지 비활성화 (정보 노출 방지)
DisableHTMLErrors: true,
// 제한적인 공개 게이트웨이 설정
PublicGateways: map[string]*gateway.PublicGateway{
```

```
"secure.example.com": {
Paths: []string{"/ipfs"}, // IPNS 제외
UseSubdomains: false,
NoDNSLink: true,
DeserializedResponses: false, // Trustless만 허용
},
},
}
gwHandler := gateway.NewHandler(config, backend)
// 보안 헤더 추가
secureHandler := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
// 보안 헤더 설정
w.Header().Set("X-Content-Type-Options", "nosniff")
w.Header().Set("X-Frame-Options", "DENY")
w.Header().Set("X-XSS-Protection", "1; mode=block")
w.Header().Set("Referrer-Policy", "strict-origin-when-cross-origin")
w.Header().Set("Content-Security-Policy", "default-src 'none'; img-src 'self'; s
// HTTPS 강제 (프로덕션 환경)
if  r.Header.Get("X-Forwarded-Proto") != "https" && r.Host != "localhost" {
http.Redirect(w, r, "https://"+r.Host+r.RequestURI, http.StatusMovedPermanently)
return
}
gwHandler.ServeHTTP(w, r)
```

```
})
return secureHandler
}
```

캐싱 및 성능 최적화

```
func createOptimizedGateway(backend gateway.IPFSBackend) http.Handler {
config := gateway.Config{
DeserializedResponses: true,
RetrievalTimeout: 60 * time.Second, // 더 긴 타임아웃
MaxConcurrentRequests: 2000, // 더 많은 동시 요청
}
gwHandler := gateway.NewHandler(config, backend)
// 캐싱 미들웨어 추가
cachingHandler := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
// 정적 자산에 대한 긴 캐시 설정
if strings.HasSuffix(r.URL.Path, ".css") ||
strings.HasSuffix(r.URL.Path, ".js") ||
strings.HasSuffix(r.URL.Path, ".png") {
w.Header().Set("Cache-Control", "public, max-age=31536000") // 1년
}
// IPFS 콘텐츠에 대한 불변 캐시 설정
```

```
if strings.HasPrefix(r.URL.Path, "/ipfs/") {
w.Header().Set("Cache-Control", "public, max-age=29030400, immutable") // 1년, 불
}
// 압축 지원
if strings.Contains(r.Header.Get("Accept-Encoding"), "gzip") {
w.Header().Set("Content-Encoding", "gzip")
}
gwHandler.ServeHTTP(w, r)
})
return cachingHandler
}
```

실제 예제: 완전한 Gateway 서버

```
package main

import (
"context"
"flag"
"log"
"net/http"
"os"
"os/signal"
```

```
"strconv"
"syscall"
"time"
"github.com/ipfs/boxo/blockservice"
"github.com/ipfs/boxo/blockstore"
"github.com/ipfs/boxo/exchange/offline"
"github.com/ipfs/boxo/gateway"
"github.com/ipfs/go-datastore"
dsync  "github.com/ipfs/go-datastore/sync"
"github.com/prometheus/client_golang/prometheus/promhttp"
)

func  main() {
var (
port = flag.Int("port", 8080, "서버 포트")
trustless = flag.Bool("trustless", false, "Trustless 모드 활성화")
timeout = flag.Duration("timeout", 30*time.Second, "검색 타임아웃")
maxReqs = flag.Int("max-requests", 1000, "최대 동시 요청 수")
)
flag.Parse()
ctx, cancel := context.WithCancel(context.Background())
defer  cancel()
// 블록스토어 설정
```

```
ds := dsync.MutexWrap(datastore.NewMapDatastore())

bs := blockstore.NewBlockstore(ds)

blockService := blockservice.New(bs, offline.Exchange(bs))

// Gateway 백엔드 생성

backend, err := gateway.NewBlocksBackend(blockService)

if  err != nil {

log.Fatal("백엔드 생성 실패:", err)

}

// Gateway 설정

config := gateway.Config{

DeserializedResponses: !*trustless,

NoDNSLink: *trustless,

RetrievalTimeout: *timeout,

MaxConcurrentRequests: *maxReqs,

PublicGateways: map[string]*gateway.PublicGateway{

"localhost": {

Paths: []string{"/ipfs", "/ipns"},

UseSubdomains: true,

DeserializedResponses: !*trustless,

},

},

}

// Gateway 핸들러 생성
```

```
gwHandler := gateway.NewHandler(config, backend)

// HTTP 서버 설정

mux := http.NewServeMux()

mux.Handle("/ipfs/", gwHandler)

mux.Handle("/ipns/", gwHandler)

mux.Handle("/metrics", promhttp.Handler())

// 헬스체크 엔드포인트

mux.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {

w.WriteHeader(http.StatusOK)

w.Write([]byte("OK"))

})

// 호스트명 핸들러로 래핑

handler := gateway.NewHostnameHandler(config, backend, mux)

// CORS 헤더 추가

handler = gateway.NewHeaders(nil).ApplyCors().Wrap(handler)

// HTTP 서버 생성

server := &http.Server{

Addr: ":" + strconv.Itoa(*port),

Handler: handler,

ReadTimeout: 30 * time.Second,

WriteTimeout: 30 * time.Second,

IdleTimeout: 120 * time.Second,

}
```

```
// 서버 시작
go func() {
log.Printf("Gateway 서버 시작: http://localhost:%d", *port)
log.Printf("Trustless 모드: %v", *trustless)
log.Printf("메트릭: http://localhost:%d/metrics", *port)
log.Printf("헬스체크: http://localhost:%d/health", *port)
if err := server.ListenAndServe(); err != nil && err != http.ErrServerClosed {
log.Fatal("서버 시작 실패:", err)
}
}()
// 우아한 종료 처리
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("서버 종료 중...")
shutdownCtx, shutdownCancel := context.WithTimeout(context.Background(), 30*time
defer shutdownCancel()
if err := server.Shutdown(shutdownCtx); err != nil {
log.Fatal("서버 종료 실패:", err)
}
log.Println("서버가 정상적으로 종료되었습니다")
}
```

모범 사례

1. 보안 고려사항

- 프로덕션 환경에서는 Trustless 모드 사용 권장
- DNSLink 기능은 신뢰할 수 있는 환경에서만 활성화
- 적절한 타임아웃과 요청 제한 설정

2. 성능 최적화

- 적절한 캐시 헤더 설정으로 CDN 활용
- 동시 요청 수 제한으로 리소스 보호
- 메트릭 모니터링으로 성능 추적

3. 운영 관리

- 헬스체크 엔드포인트 제공
- 구조화된 로깅 구현
- 우아한 종료 처리

4. 확장성

- 로드 밸런서 뒤에서 여러 인스턴스 실행
- 상태 비저장 설계로 수평 확장 가능
- 메트릭 기반 자동 스케일링

문제 해결

일반적인 오류와 해결책

1. **404 Not Found 오류**

- 콘텐츠가 실제로 존재하는지 확인
- 백엔드 연결 상태 점검
- 경로 형식이 올바른지 확인

2. **504 Gateway Timeout**

- RetrievalTimeout 설정 확인
- 네트워크 연결 상태 점검
- 백엔드 성능 모니터링

3. **429 Too Many Requests**

- MaxConcurrentRequests 설정 조정
- 클라이언트 요청 패턴 분석
- 로드 밸런싱 고려

4. **CORS 오류**

- CORS 헤더 설정 확인
- 브라우저 개발자 도구로 헤더 검사
- 프리플라이트 요청 처리 확인

5. **메모리 사용량 증가**

- 블록스토어 크기 모니터링
- 가비지 컬렉션 최적화
- 메모리 프로파일링 수행